ONLINE MASTERS IN **DATA SCIENCE**

DSC 258 – NATURAL LANGUAGE PROCESSING

NEURAL LANGUAGE MODELS

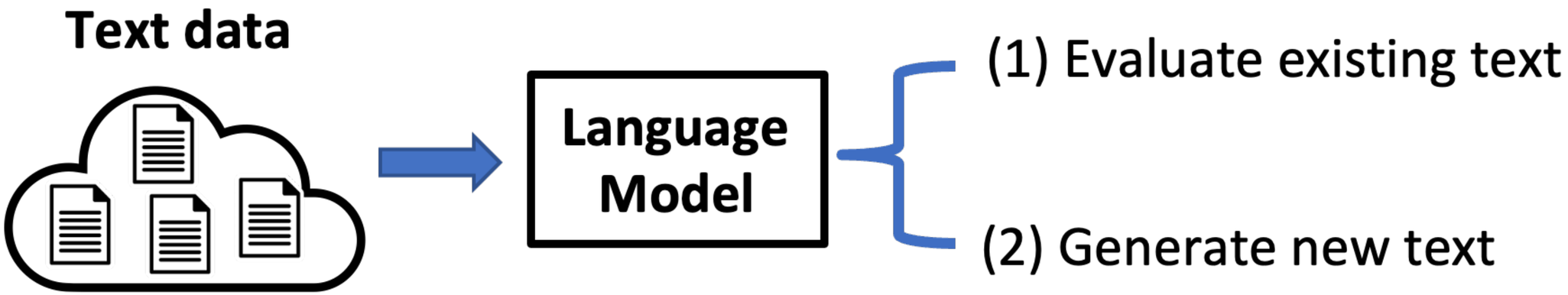
UC San Diego

COMPUTER SCIENCE & ENGINEERING

HALICIOĞLU DATA SCIENCE INSTITUTE

Recurrent Language Models

What are language models and what are they used for?



Recurrent Language Models

Use chain rule to generate words left-to-right

$$P(w_1 \dots w_n) = \prod_i P(w_i | w_1 \dots w_{i-1})$$

Conditioning on the entire left context is difficult

P(??? | Turn to page 134 and look at the picture of the)

N-gram models make a Markov assumption

$$P(w_1 \dots w_n) = \prod_i P(w_i | w_{i-k} \dots w_{i-1})$$

$$P(\text{please close the door}) =$$

$$P(\text{please}|\text{START})P(\text{close}|\text{please}) \dots P(\text{STOP}|door)$$

Recurrent Language Models

Use chain rule to generate words left-to-right

$$P(w_1 \dots w_n) = \prod_i P(w_i | w_1 \dots w_{i-1})$$

Conditioning on the entire left context is difficult

P(??? | Turn to page 134 and look at the picture of the)

What if we didn't make a Markov assumption?

- Good news: Language model can track long-range interactions between words

  The **reign** has lasted as long as the life of the **queen**.

  The **rain** has lasted as long as the life of the **clouds**.

- Bad news: If done naively, way too many parameters to learn, sparsity

$p(w_n | w_1, \dots, w_{n-1})$ is a table of size $|V|^n$ where V is the vocabulary

Recurrent Language Models

What if we didn't make a Markov assumption?

- Good news: Language model can track long-range interactions between words
- Bad news: If done naively, way too many parameters to learn, sparsity

$p(w_n|w_1, \dots, w_{n-1})$ is a table of size $|V|^n$ where V is the vocabulary

Solution:

- Use neural nets to parameterize the conditional distribution on next word!
- Appropriate neural architecture will allow for *parameter sharing* between all conditional next-word distributions
- This can result in fewer parameter overall
- This can help the model deal with sparsity

Recurrent Language Models

Outline:

- **Warm-up:** *Neural* n-gram language model – make the Markov assumption, but use a neural net
- Introduce recurrent neural language model – break the Markov assumption, use a neural net
- Sampling and training – show how to train and generate using a neural language model

Recurrent Language Models

- Markov assumption:

$$p(x_1, \ldots, x_n) = \prod_i p(x_i \mid x_{i-k}, \ldots, x_{i-1})$$

- Chain-rule (no assumptions):

$$p(x_1, \ldots, x_n) = \prod_i p(x_i \mid x_1, \ldots, x_{i-1})$$

- But what about number of params / smoothing?

$$p(x_i \mid x_1, \ldots, x_{i-1})$$